LAPORAN UAS
Re-Engineering Basis Data SentraMart

Keterangan	Isi
Mata Kuliah	Advanced Database Systems (UAS Blok A ,Oktober 2026)
Program Studi	Sains Data
Nama / NIM	Uswatun Hasanah / 25110500030

1. Ringkasan

Pada promo 12.12, database SentraMart (skema **legacy**) bermasalah: stok bisa minus (overselling), pembayaran tertagih dua kali, query lambat, dan keamanan lemah. Audit pada database menunjukkan temuan berikut:

- **43 produk** bernilai stok minus dengan total **771 unit**.
- **515 order** terkena pembayaran ganda dengan total kelebihan tagih **Rp 306.878.100**.
- Penulisan status order tidak seragam (huruf kecil/campuran) pada **1.200 baris**.
- Total pesanan tidak cocok dengan rincian item (selisih terbesar sekitar **Rp 43.500** per order) dan skema legacy **tidak punya foreign key**.

Solusinya: membuat **skema v2** di PostgreSQL (tipe NUMERIC, CHECK, FOREIGN KEY, UNIQUE, partisi tabel orders), menambah index, membuat stored procedure anti-overselling/anti-double-charge/anti-deadlock, memindahkan katalog produk dan log aktivitas ke **MongoDB**, serta menerapkan role, Row-Level Security, dan masking data pelanggan. Migrasi data berhasil: seluruh tabel sama jumlahnya, kecuali tabel payments yang berkurang **515 baris** karena duplikat dibuang.

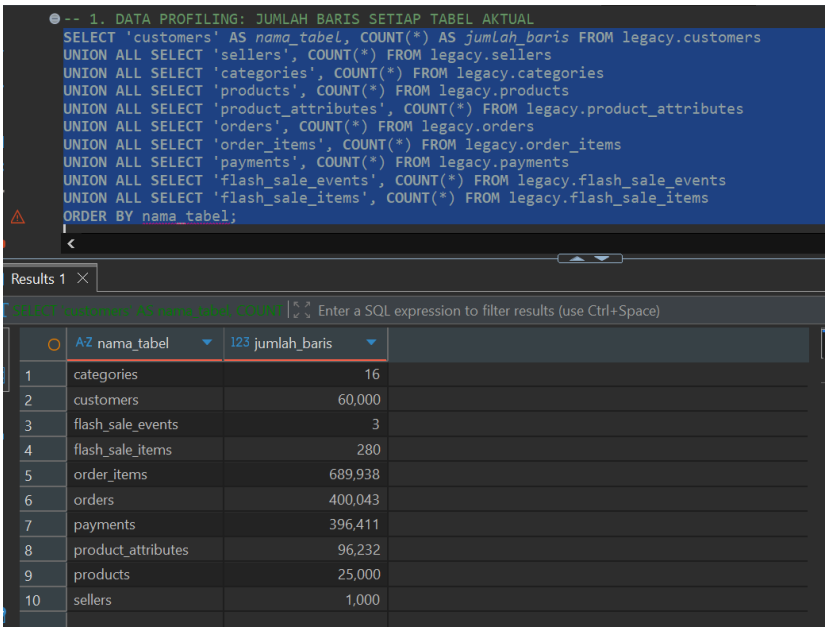
2. Task 1 ,Audit Database Legacy

2.1 Jumlah data

Jumlah baris setiap tabel dihitung dengan query UNION ALL pada skema legacy (Gambar 1).

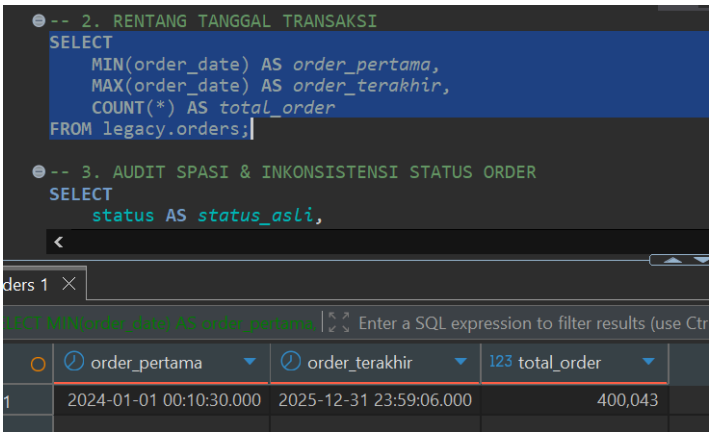
Tabel	Jumlah baris	Keterangan
customers	60.000	Data pelanggan
sellers	1.000	Merchant terdaftar
categories	16	Kategori produk
products	25.000	Produk beserta stok
product_attributes	96.232	Atribut produk (model EAV)
orders	400.043	Pesanan 2024–2025
order_items	689.938	Rincian item pesanan

Tabel	Jumlah baris	Keterangan
payments	396.411	Riwayat pembayaran
flash_sale_events	3	Event flash sale
flash_sale_items	280	Produk peserta flash sale



Gambar 1. Hasil hitung baris semua tabel legacy

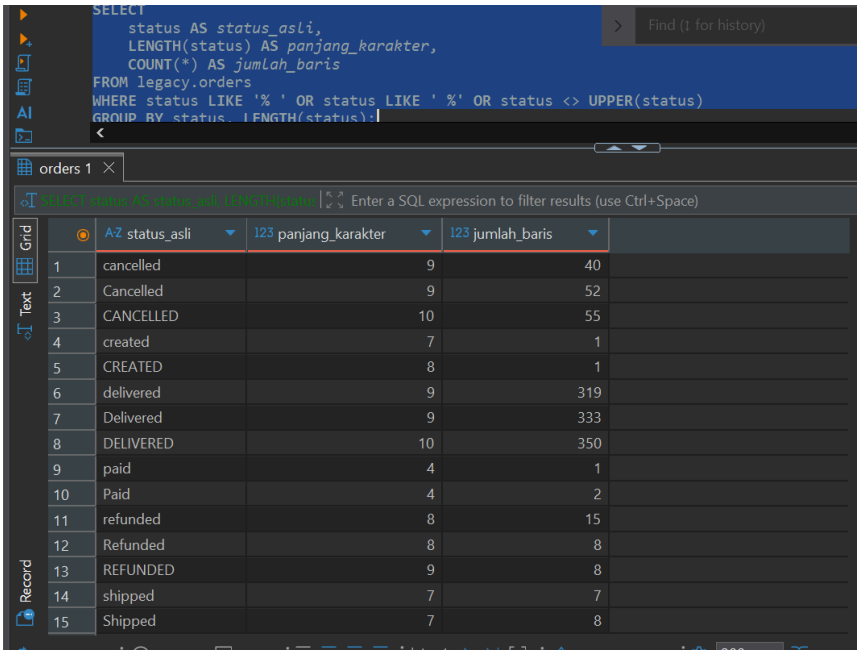
Rentang tanggal transaksi: pesanan pertama **1 Januari 2024 pukul 00:10:30** dan terakhir **31 Desember 2025 pukul 23:59:06**, total 400.043 order (Gambar 2).



Gambar 2. Rentang tanggal transaksi pada tabel orders

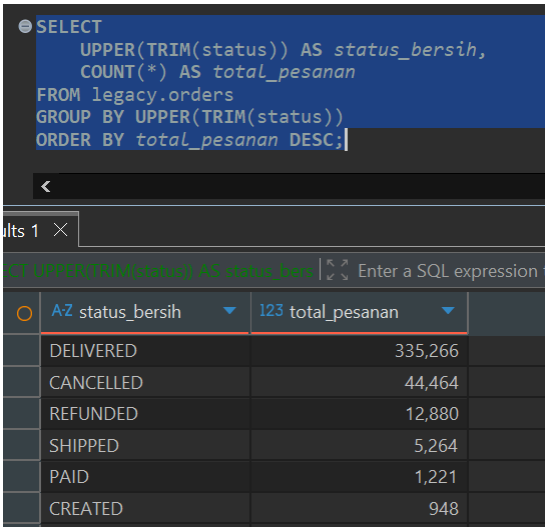
2.2 Status order tidak seragam

Kolom orders.status ditulis dengan berbagai bentuk huruf, misalnya cancelled, Cancelled, dan CANCELLED. Query pada Gambar 3 menemukan 15 variasi penulisan yang mencakup **1.200 baris**. Setelah dinormalisasi dengan UPPER(TRIM(status)), tersisa 6 status yang bersih (Gambar 4).



Gambar 3. Variasi penulisan status order pada skema legacy

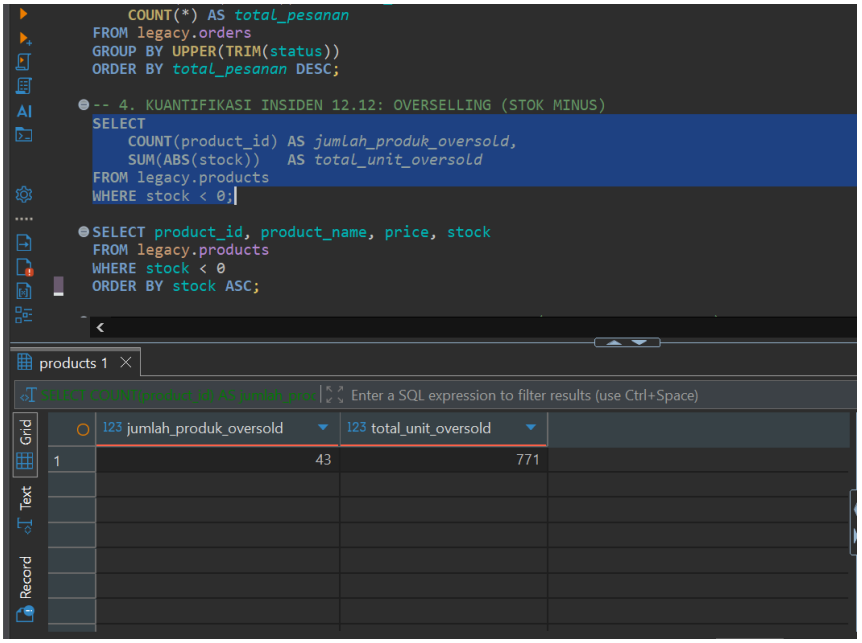
Status (setelah dibersihkan)	Jumlah order
DELIVERED	335.266
CANCELLED	44.464
REFUNDED	12.880
SHIPPED	5.264
PAID	1.221
CREATED	948
Total	400.043



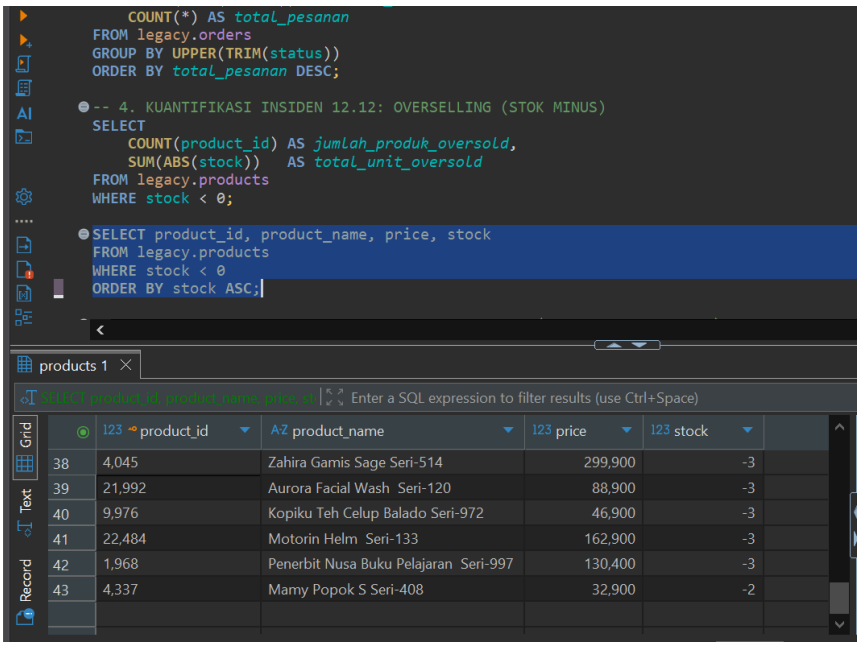
Gambar 4. Jumlah order per status setelah UPPER(TRIM(status))

2.3 Dampak insiden 12.12

a. Overselling. Query menghitung produk dengan stok di bawah nol. Hasilnya **43 produk** dengan total **771 unit** minus (Gambar 5). Contoh produknya terlihat pada Gambar 6. Penyebabnya, kolom stock tidak punya batasan CHECK (stock >= 0) dan tidak ada penguncian saat checkout.

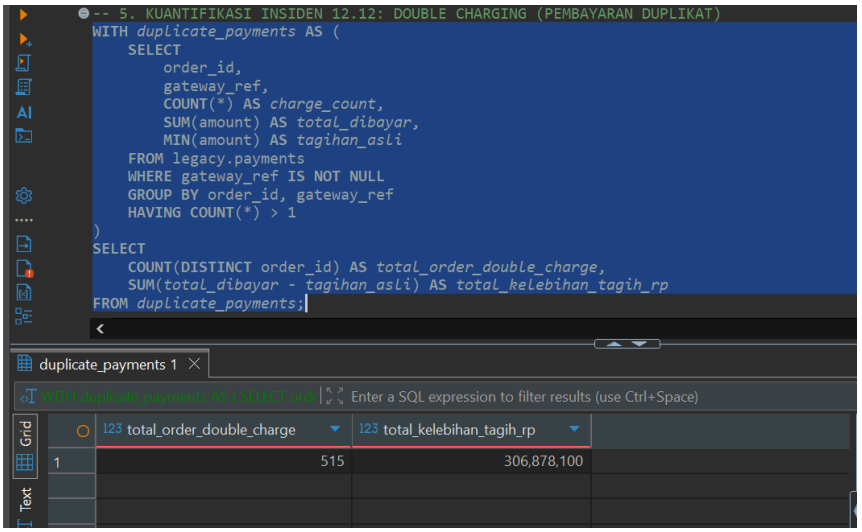


Gambar 5. Jumlah produk oversold dan total unit minus



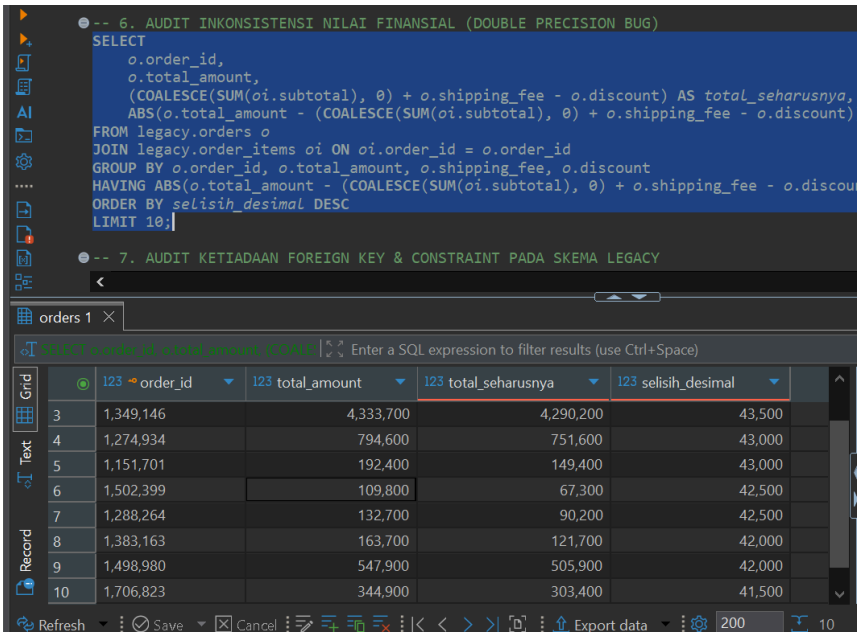
Gambar 6. Daftar produk yang stoknya minus

b. Double charging. Pembayaran dengan gateway_ref yang sama dikelompokkan per order. Hasilnya **515 order** tertagih lebih dari sekali dengan total kelebihan tagih **Rp 306.878.100** (Gambar 7). Penyebabnya, callback gateway yang dikirim ulang diproses lagi karena gateway_ref tidak UNIQUE.



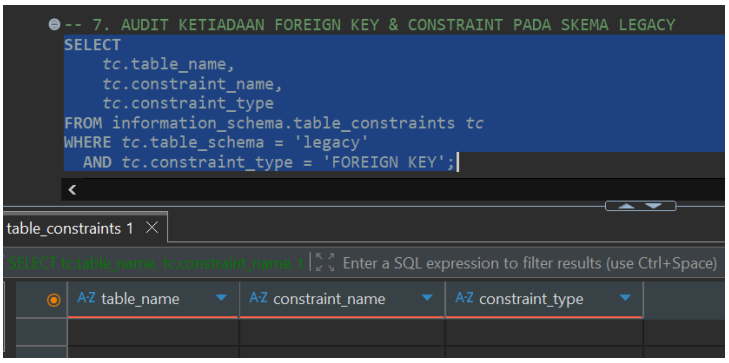
Gambar 7. Audit pembayaran ganda

c. Total pesanan tidak konsisten. Total order dibandingkan dengan jumlah subtotal item ditambah ongkir dikurangi diskon. Sepuluh selisih terbesar berkisar antara Rp 41.500 sampai lebih dari Rp 43.000 per order (Gambar 8). Artinya data keuangan sulit direkonsiliasi.



Gambar 8. Sepuluh selisih terbesar antara total_amount dan hasil hitung ulang

d. Tidak ada foreign key. Query ke information_schema untuk constraint FOREIGN KEY pada skema legacy mengembalikan hasil **kosong** (Gambar 9), sehingga data yatim bisa terjadi.



Gambar 9. Pengecekan foreign key skema legacy (hasil kosong)

2.4 Kelemahan skema legacy dan kebutuhan sistem baru

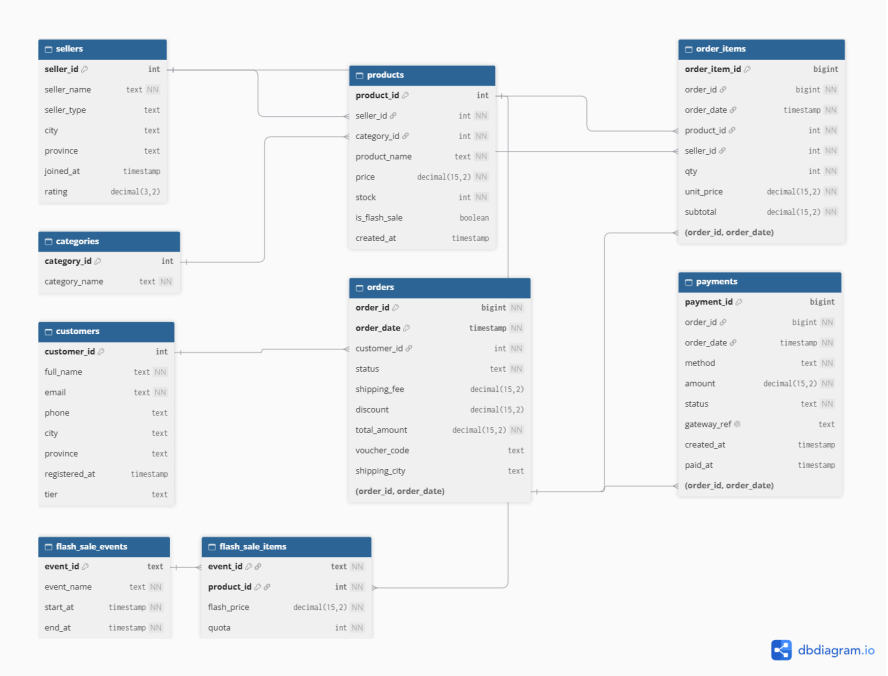
No	Kelemahan	Dampak
1	Nilai uang memakai tipe floating point	Angka keuangan tidak presisi dan sulit direkonsiliasi
2	Stok tanpa CHECK (stock >= 0)	Overselling: 43 produk, 771 unit minus
3	gateway_ref tidak UNIQUE	Double charge: 515 order, Rp 306.878.100
4	Tidak ada foreign key (terbukti Gambar 9)	Data yatim dan inkonsistensi
5	Tidak ada index selain primary key	Query besar harus Seq Scan (lihat Task 3)
6	Katalog produk model EAV (96.232 baris)	Pencarian spesifikasi butuh banyak self-join

Kebutuhan sistem baru: (1) checkout atomik yang tidak bisa membuat stok minus, (2) callback pembayaran yang aman bila dikirim berulang, (3) akses data dibatasi sesuai peran, dan (4) katalog fleksibel yang cepat dicari.

3. Task 2 ,Desain dan Implementasi Skema v2

3.1 ERD

ERD skema v2 dirancang di dbdiagram.io (Gambar 10). Tabel orders memakai primary key gabungan (order_id, order_date) karena dipartisi berdasarkan order_date.



Gambar 10. ERD skema v2

3.2 Implementasi skema

Masalah legacy	Solusi di skema v2
Nilai uang float	Semua kolom uang memakai NUMERIC(15,2)
Stok minus	CHECK (stock >= 0) pada v2.products
Pembayaran ganda	gateway_ref bersifat UNIQUE pada v2.payments
Data yatim	FOREIGN KEY di semua tabel anak
Status tidak seragam	CHECK status IN (CREATED, PAID, SHIPPED, DELIVERED, CANCELLED, REFUNDED)
Tabel orders besar	Range Partitioning per tahun: orders_2024 dan orders_2025

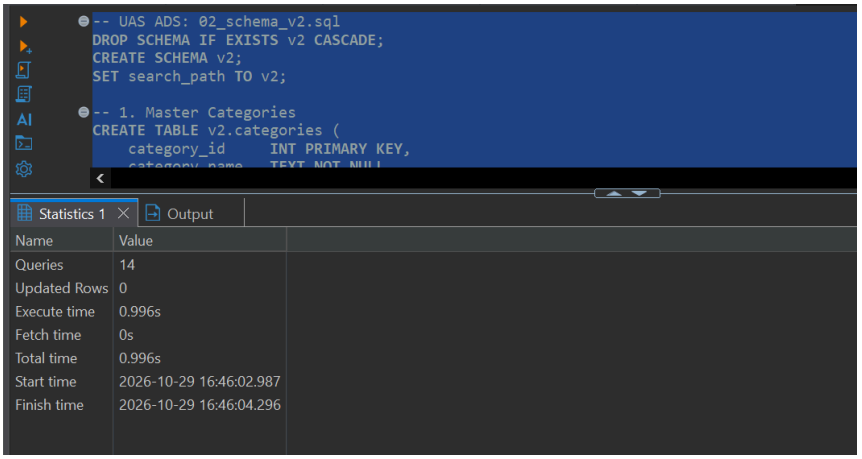
Cuplikan DDL (skrip lengkap ada di sql/02_schema_v2.sql):

```
CREATE TABLE v2.products (
  product_id  INT PRIMARY KEY,
  seller_id   INT NOT NULL REFERENCES v2.sellers(seller_id),
  category_id INT NOT NULL REFERENCES v2.categories(category_id),
  product_name TEXT NOT NULL,
  price     NUMERIC(15,2) NOT NULL CHECK (price >= 0),
  stock     INT NOT NULL CHECK (stock >= 0),  -- cegah overselling
  ...
);

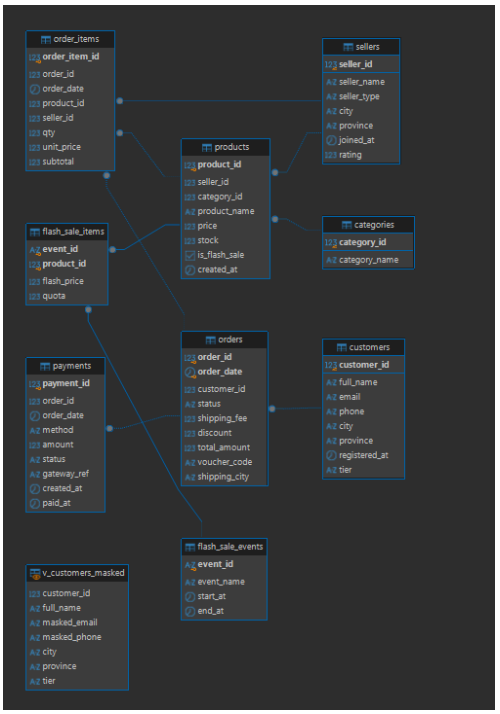
CREATE TABLE v2.orders ( ... PRIMARY KEY (order_id, order_date) )
  PARTITION BY RANGE (order_date);
CREATE TABLE v2.orders_2024 PARTITION OF v2.orders
  FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
CREATE TABLE v2.orders_2025 PARTITION OF v2.orders
  FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');

CREATE TABLE v2.payments ( ... gateway_ref TEXT UNIQUE,  -- cegah double charge
  FOREIGN KEY (order_id, order_date) REFERENCES v2.orders(order_id, order_date) );
```

Skrip dijalankan di DBeaver dan berhasil (Gambar 11). Struktur tabel v2 beserta relasinya terlihat pada diagram DBeaver (Gambar 12).



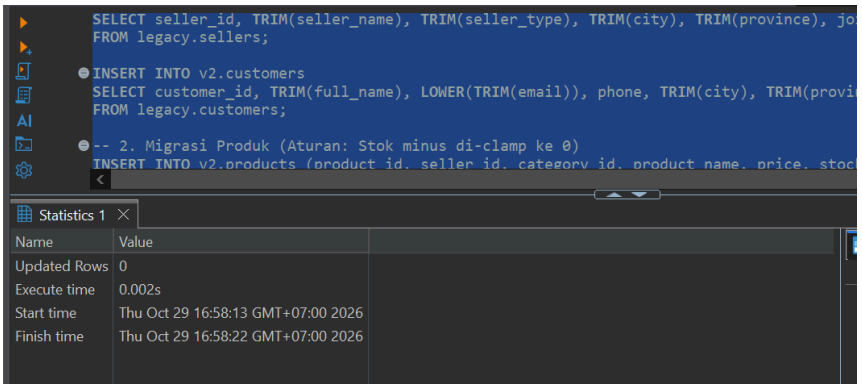
Gambar 11. Eksekusi skrip 02_schema_v2.sql



Gambar 12. Diagram tabel skema v2 di DBeaver

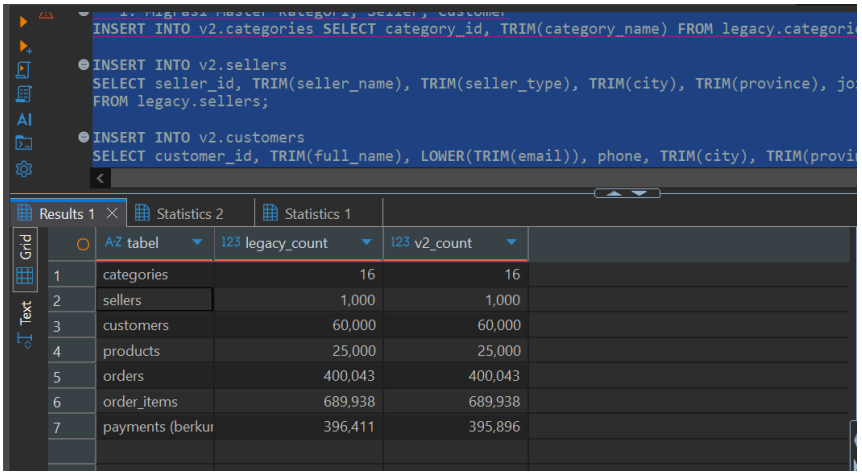
3.3 Migrasi data

Data dipindahkan dari legacy ke v2 dengan pembersihan: TRIM pada teks, email dijadikan huruf kecil, stok minus diubah menjadi 0, status dinormalisasi dengan UPPER(TRIM()), dan pembayaran duplikat (gateway_ref sama) hanya disimpan yang pertama. Hasil migrasi dapat dilihat pada Gambar 13 dan 14.



Gambar 13. Eksekusi skrip 03_migration.sql

Tabel	Legacy	Skema v2	Selisih
categories	16	16	0
sellers	1.000	1.000	0
customers	60.000	60.000	0
products	25.000	25.000	0
orders	400.043	400.043	0
order_items	689.938	689.938	0
payments	396.411	395.896	-515 (duplikat dibuang)



Gambar 14. Rekonsiliasi jumlah baris legacy vs v2

Selisih 515 baris pada payments sama dengan jumlah order double charge yang ditemukan pada audit (Gambar 7), sehingga pembersihan duplikat berjalan sesuai.

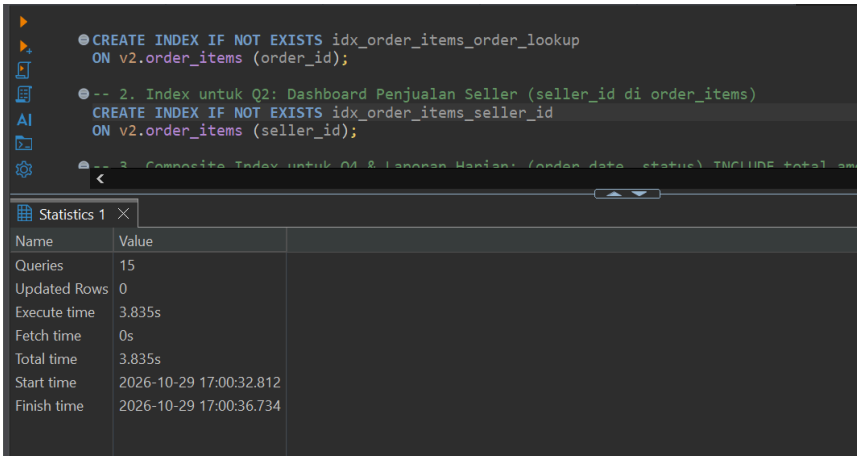
4. Task 3 ,Optimasi Query

4.1 Index yang dibuat

```
CREATE INDEX idx_orders_customer_date ON v2.orders (customer_id, order_date DESC);
CREATE INDEX idx_order_items_order_lookup ON v2.order_items (order_id);
CREATE INDEX idx_order_items_seller_id ON v2.order_items (seller_id);
CREATE INDEX idx_orders_date_status_covering ON v2.orders (order_date, status)
  INCLUDE (total_amount);
```

```
CREATE INDEX idx_order_items_product_id ON v2.order_items (product_id);
CREATE INDEX idx_payments_pending_created ON v2.payments (status, created_at)
  WHERE status = 'PENDING';  -- partial index
CREATE INDEX idx_products_flash_active ON v2.products (product_id, price, stock)
  WHERE is_flash_sale = TRUE AND stock > 0;  -- partial index
```

Skrip index berhasil dijalankan (Gambar 15).

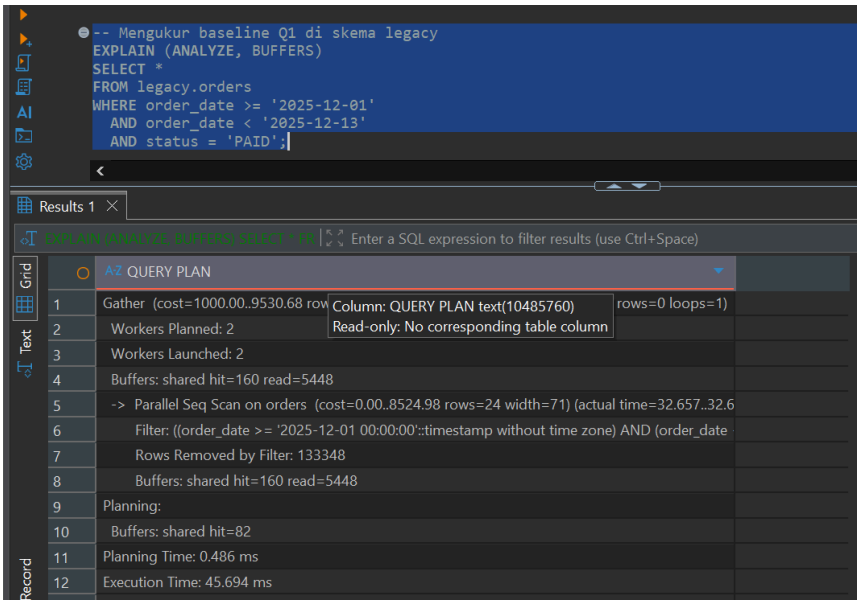


Gambar 15. Eksekusi skrip 04_optimization.sql

4.2 Baseline query Q1 pada skema legacy

Query Q1 mengambil order berstatus PAID pada 1–12 Desember 2025, diukur dengan EXPLAIN (ANALYZE, BUFFERS) pada skema legacy (Gambar 16).

Komponen plan	Hasil
Jenis scan	Parallel Seq Scan on orders (2 worker)
Rows Removed by Filter	133.348
Buffers	shared hit=160, read=5.448
Planning Time	0,486 ms
Execution Time	45,694 ms



Gambar 16. EXPLAIN ANALYZE Q1 pada skema legacy

Analisis: karena tidak ada index pada order_date dan status, PostgreSQL membaca seluruh tabel (Seq Scan) lalu membuang banyak baris yang tidak cocok. Pada v2, index idx_orders_date_status_covering disiapkan agar query seperti ini cukup memakai index scan.

5. Task 4 ,Transaksi dan Konkurensi

5.1 Anomali pada legacy dan solusinya di v2

Anomali	Penyebab di legacy	Solusi v2
Overselling	Dua sesi membaca stok yang sama sebelum salah satu mengurangi (lost update)	v2.checkout_safe: kunci baris produk dengan SELECT ... FOR UPDATE, cek stok, baru kurangi
Double charge	Callback gateway yang sama diproses dua kali	v2.payment_callback_safe: INSERT ... ON CONFLICT (gateway_ref) DO NOTHING
Deadlock	Dua keranjang mengunci produk dengan urutan berbeda (101→202 dan 202→101)	v2.checkout_cart_safe: id produk selalu dikunci berurutan ASC

5.2 Stored procedure v2

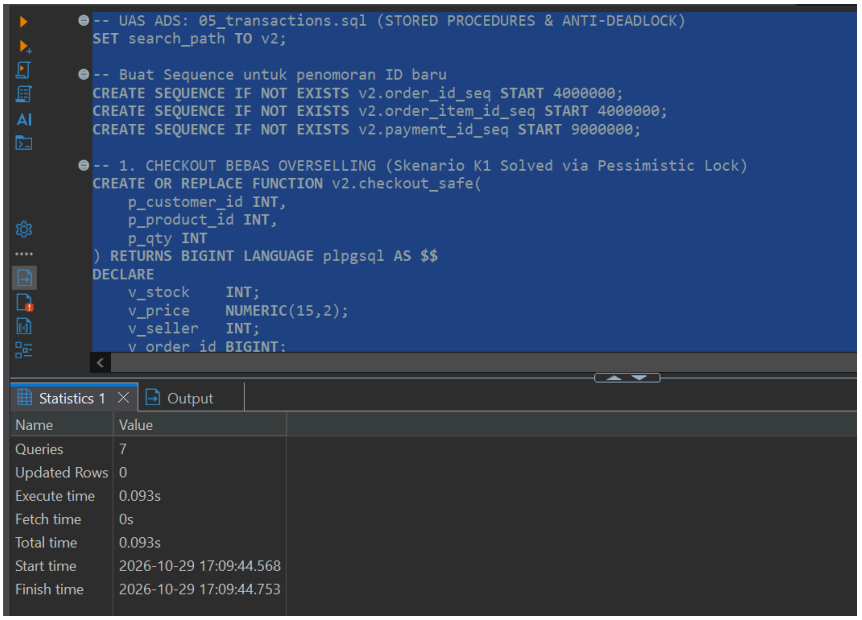
Cuplikan inti dari sql/05_transactions.sql:

```
-- checkout_safe: kunci baris produk lalu cek stok
SELECT stock, price, seller_id INTO v_stock, v_price, v_seller
FROM v2.products WHERE product_id = p_product_id FOR UPDATE;
IF v_stock < p_qty THEN RAISE EXCEPTION 'Stok tidak cukup'; END IF;
UPDATE v2.products SET stock = stock - p_qty WHERE product_id = p_product_id;

-- payment_callback_safe: callback ganda diabaikan
INSERT INTO v2.payments (...) VALUES (...)
ON CONFLICT (gateway_ref) DO NOTHING RETURNING payment_id INTO v_inserted_id;
```

```
-- checkout_cart_safe: kunci produk berurutan agar tidak deadlock
FOR v_pid IN SELECT unnest(p_product_ids) AS pid ORDER BY pid ASC LOOP
  PERFORM stock FROM v2.products WHERE product_id = v_pid FOR UPDATE;
  UPDATE v2.products SET stock = stock - 1 WHERE product_id = v_pid;
END LOOP;
```

Skrip dijalankan dan membuat 3 sequence serta 3 fungsi (Gambar 17).



Gambar 17. Eksekusi skrip 05_transactions.sql

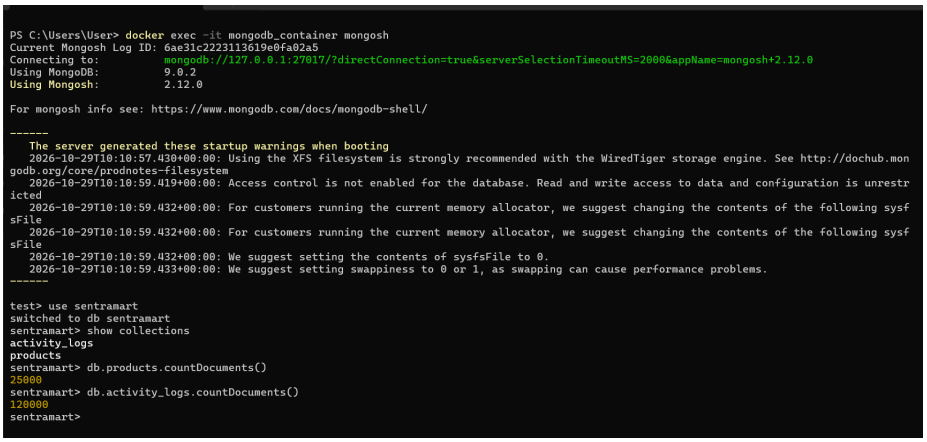
5.3 Isolation level dan WAL

- **READ COMMITTED + FOR UPDATE** dipilih daripada SERIALIZABLE. Saat flash sale banyak transaksi berebut stok yang sama; SERIALIZABLE akan membatalkan banyak transaksi (error 40001) sehingga aplikasi harus mengulang terus, sedangkan FOR UPDATE membuat transaksi antre dengan tertib.
- **WAL (Write-Ahead Logging)** menjaga sifat Durability. Setiap perubahan ditulis ke log WAL di disk sebelum transaksi dinyatakan commit. Jika server mati mendadak, PostgreSQL membaca WAL saat start ulang: transaksi yang sudah commit diulang (REDO) dan yang belum selesai dibatalkan (UNDO).

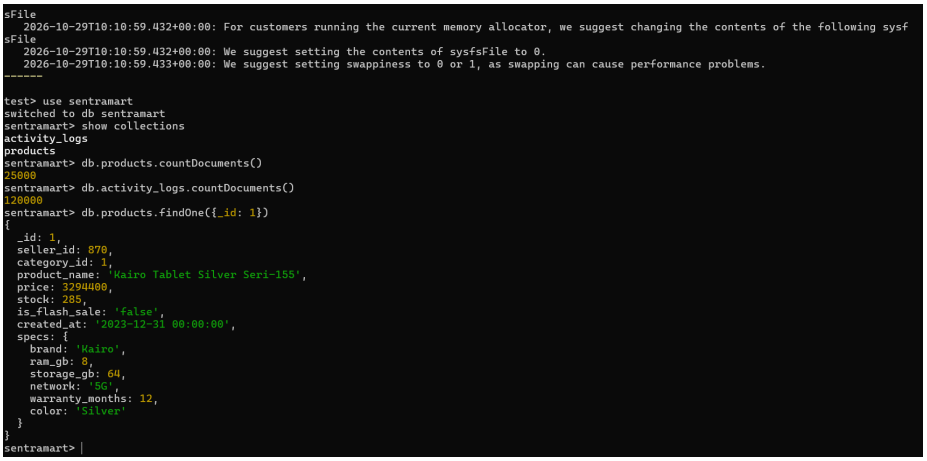
6. Task 5 ,Solusi NoSQL (MongoDB)

6.1 Data yang dipindahkan

Dua jenis data kurang cocok di tabel relasional: **katalog produk** dengan atribut berbeda-beda (96.232 baris EAV) dan **log aktivitas** pengguna yang banyak ditulis (120.000 event). Keduanya dimuat ke MongoDB (container Docker, versi 9.0.2): koleksi products berisi 25.000 dokumen dan activity_logs berisi 120.000 dokumen (Gambar 18). Atribut produk disimpan menyatu di dalam dokumen (embedding), seperti contoh pada Gambar 19.



Gambar 18. Koneksi mongosh dan jumlah dokumen tiap koleksi



Gambar 19. Contoh dokumen produk (findOne)

6.2 Query filter spesifikasi produk

Query berikut setara dengan Q3 pada model EAV (cari tablet 5G RAM 8 GB harga Rp 2–5 juta):

```
db.products.find({
 category_id: 1, "specs.ram_gb": 8, "specs.network": "5G",
 price: { $gte: 2000000, $lte: 5000000 }, stock: { $gt: 0 }
}).sort({ price: 1 }).limit(50).explain("executionStats")
```

Hasil executionStats	Nilai
nReturned	50 dokumen
totalKeysExamined / totalDocsExamined	56 / 50
Tahapan plan	LIMIT → FETCH → IXSCAN
Index yang dipakai	compound index category_id, specs.ram_gb, specs.network, price, stock
executionTimeMillis	14 ms

Query memakai **IXSCAN** dan hanya memeriksa 56 key untuk menghasilkan 50 dokumen, jadi tidak perlu join dan tidak membaca seluruh 25.000 dokumen (Gambar 20; output lengkap ada di Lampiran A).



Gambar 20. Bagian executionStats hasil explain

6.3 Analisis log aktivitas

Log aktivitas mencakup **1 November ,31 Desember 2025** (Gambar 21). Terdapat **4.841** event apply_voucher, dan **1.479** di antaranya gagal. Rincian penyebab kegagalan terlihat pada Gambar 22.



Gambar 21. Contoh log apply_voucher dan rentang tanggal log

Kode error voucher	Jumlah
MIN_SPEND (belum memenuhi minimum belanja)	519
QUOTA_EXCEEDED (kuota voucher habis)	488
EXPIRED (voucher kedaluwarsa)	472
Total gagal	1.479



Gambar 22. Jumlah event voucher dan agregasi error_code

6.4 Pembagian data PostgreSQL dan MongoDB

Aspek	PostgreSQL EAV (legacy)	MongoDB
Struktur katalog	96.232 baris key-value, butuh banyak self-join	Satu dokumen berisi semua spesifikasi
Skema atribut	Kaku	Fleksibel per produk
Cocok untuk	Data transaksi dan uang	Katalog dan log aktivitas

Keputusan: data keuangan, stok, dan pembayaran tetap di PostgreSQL agar konsisten (ACID). Katalog produk dan log aktivitas dipindahkan ke MongoDB. Untuk meredam lonjakan flash sale, kuota promo dapat ditaruh di **Redis** sebagai counter (DECRBY atomik); hanya request yang mendapat slot yang diteruskan ke v2.checkout_safe.

7. Task 6 ,Arsitektur dan Keamanan

7.1 Arsitektur target

Alur: Klien web/mobile → Load Balancer / API Gateway → tiga layanan (Checkout, Catalog, Reporting).

- Checkout Service memakai **Redis** (counter flash sale) dan **PgBouncer** menuju **PostgreSQL Primary**.
- Catalog Service membaca **MongoDB** (spesifikasi produk dan log aktivitas).

- Reporting Service membaca **PostgreSQL Read Replica** (streaming replication) agar laporan berat tidak mengganggu checkout.
- Primary mengarsipkan WAL secara kontinu dengan **pgBackRest** ke S3/MinIO untuk backup dan Point-in-Time Recovery.

7.2 Skalabilitas, RPO dan RTO

- Partisi orders ditingkatkan menjadi bulanan (pg_partman) saat volume naik 3×.
- PgBouncer (transaction pooling) menggabungkan banyak koneksi aplikasi menjadi sedikit koneksi ke database.
- MongoDB di-shard bila activity_logs sangat besar, memakai shard key hashed session_id.

Parameter	Target	Cara mencapainya
RPO (data hilang maksimal)	< 1 detik	WAL archiving kontinu (pgBackRest) ke storage terpisah
RTO (waktu pemulihan)	< 30 detik	Failover otomatis dengan Patroni + etcd, replica dipromosikan jadi primary

7.3 Keamanan

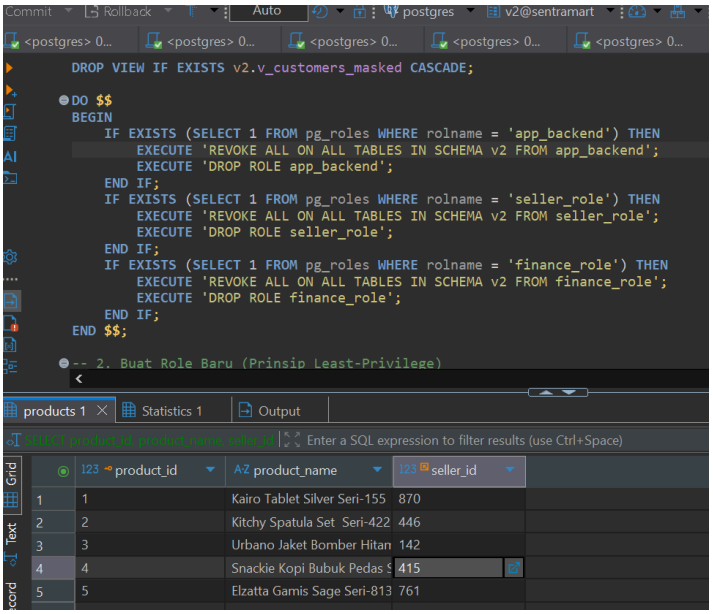
Keamanan dibuat dengan prinsip **least privilege** (sql/06_security.sql):

Role	Hak akses
app_backend	SELECT/UPDATE products; SELECT/INSERT/UPDATE orders, order_items, payments
seller_role	SELECT products, dibatasi Row-Level Security hanya untuk produk seller sendiri
finance_role	SELECT saja pada orders, order_items, payments

```
ALTER TABLE v2.products ENABLE ROW LEVEL SECURITY;
CREATE POLICY seller_product_isolation ON v2.products FOR ALL TO seller_role
  USING (seller_id = NULLIF(current_setting('app.current_seller_id', true), '')::INT);

-- view masking data pelanggan (email dan nomor HP disamarkan)
CREATE OR REPLACE VIEW v2.v_customers_masked AS
SELECT customer_id, full_name, REGEXP_REPLACE(email, '(?<=.).(?=.*@)', '*', 'g') AS masked_email,
    CONCAT(SUBSTRING(phone,1,4), '****', SUBSTRING(phone, LENGTH(phone)-2, 3)) AS masked_phone,
    city, province, tier
FROM v2.customers;
```

Skrip keamanan dijalankan di DBeaver (Gambar 23). View v_customers_masked juga sudah terbentuk pada diagram skema v2 (Gambar 12).



Gambar 23. Eksekusi skrip 06_security.sql (pembuatan role)

7.4 CAP theorem dan roadmap

- **PostgreSQL (CP):** lebih memilih konsisten untuk data uang dan stok; saat jaringan terpisah, mutasi ditolak daripada data rusak.
- **MongoDB (AP):** lebih memilih tetap tersedia untuk katalog dan log, dengan eventual consistency.

Fase	Waktu	Kegiatan
1. Stabilisasi	Minggu 1–2	Deploy skema v2, migrasi data, stored procedure, bersihkan kasus double charge
2. Optimasi & NoSQL	Minggu 3–4	Index, MongoDB untuk katalog dan log, PgBouncer
3. High Availability	Minggu 5–6	Patroni + replica, RLS dan masking, backup PITR, uji beban 3×

8. Kesimpulan

- Audit membuktikan masalah di legacy: 43 produk stok minus (771 unit), 515 order double charge (Rp 306.878.100), status tidak seragam, dan tidak ada foreign key.
- Skema v2 menutup masalah tersebut lewat NUMERIC, CHECK, FOREIGN KEY, UNIQUE, dan partisi; migrasi data lengkap dengan 515 pembayaran duplikat dibersihkan.
- Index dan stored procedure (FOR UPDATE, ON CONFLICT, urutan kunci) disiapkan untuk mencegah overselling, double charge, dan deadlock.
- MongoDB dipakai untuk katalog dan log; query spesifikasi produk selesai dalam 14 ms dengan IXSCAN.

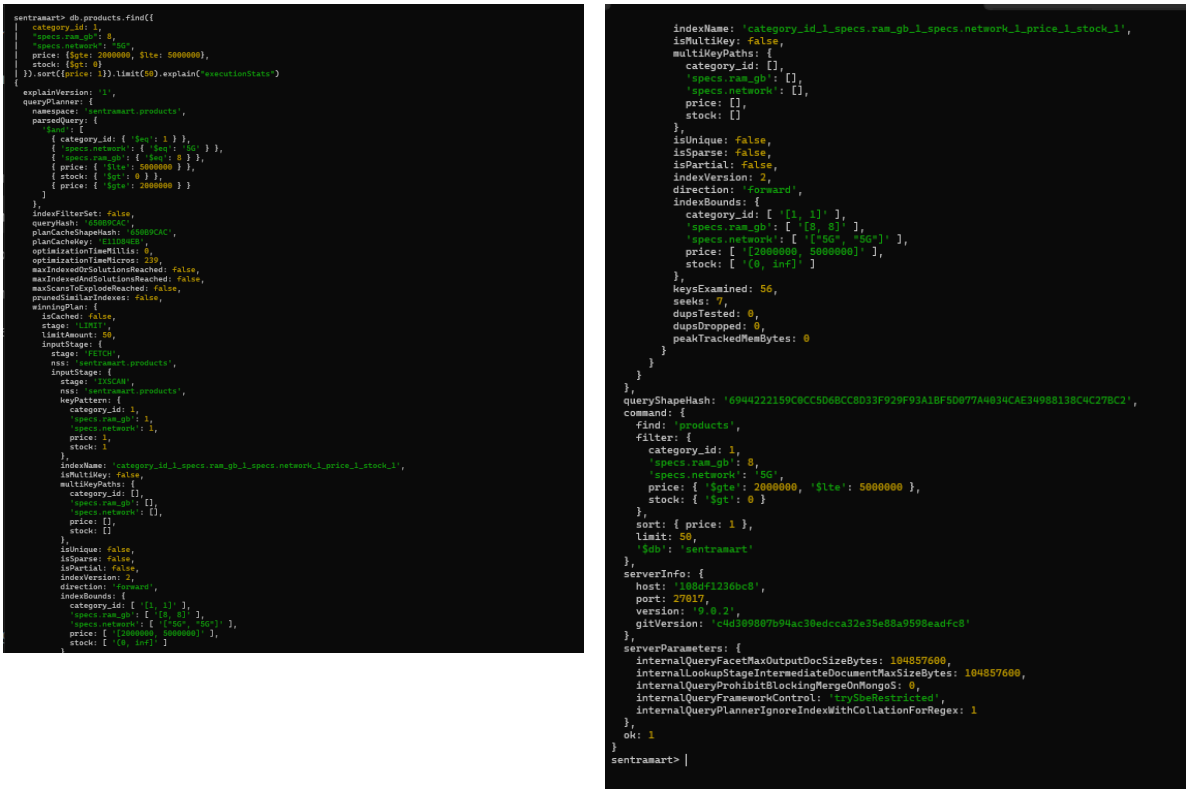
Trade-off utama: pessimistic locking membuat transaksi sedikit antre, tetapi menghindari kegagalan serialisasi; memakai dua database menambah kerja operasional, tetapi meringankan PostgreSQL dari data semi-terstruktur.

9. Pernyataan Penggunaan AI

AI Assistant digunakan sebagai bantuan teknis dalam referensi dan tempat Troubleshooting skrip SQL, merapikan format laporan, dan memberi masukan optimasi query. Seluruh angka pada laporan ini (jumlah data, produk oversold, double charge, hasil migrasi, dan hasil MongoDB) diambil dari hasil eksekusi di database SentraMart yang terlihat pada screenshot.

Lampiran A ,Output Explain MongoDB Lengkap

Output explain("executionStats") untuk query pada bagian 6.2 (bagian awal dan akhir).



Gambar 24. Output explain MongoDB (kiri: awal, kanan: akhir)

GitHub